FODS ASSIGNMENT 7

Name – Atharva Shivaji Dabkekar
Roll no. – 2090138
PRN - 1251090357
Division – S.Y A
Batch – 2
DATE – 25-08-2026 TIME – 8:00 TO 10:00

Practical 7: Non-Comparison-Based Sorting

Implement Counting Sort, Radix Sort, and Bucket Sort. Compare their efficiency with comparison-based sorting algorithms.

Answer:

1. Counting Sort:

Code:

```
#include <iostream>
using namespace std;
void countingSort(int arr[], int n) {
    if (n <= 0) return;
    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) max_val = arr[i];
    }
    int* count = new int[max_val + 1]();
    int* output = new int[n];
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }
    for (int i = 1; i <= max_val; i++) {
        count[i] += count[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
```

```
    }
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    if (n <= 0) {
        cout << "Array size must be greater than 0." << endl;
        return 1;
    }
    int* arr = new int[n];
    cout << "Enter " << n << " non-negative integers separated by spaces:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "\nOriginal array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << "\n";
    countingSort(arr, n);
    cout << "Sorted array:   ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << "\n";
    return 0;
}
```

Output:



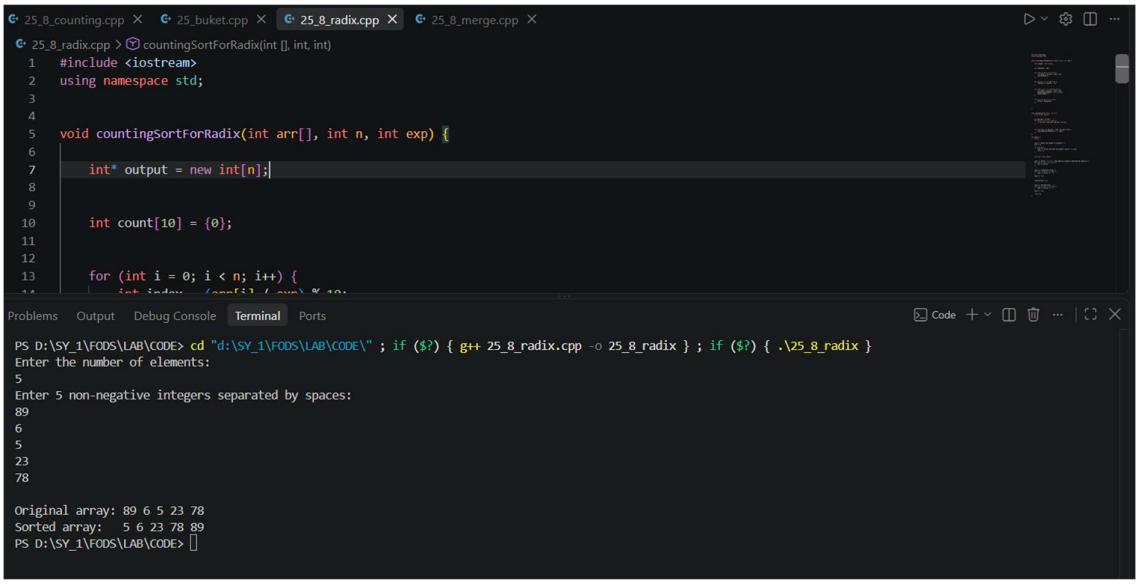
2. Radix Sort :

Code :

```
#include <iostream>
using namespace std;
void countingSortForRadix(int arr[], int n, int exp) {
    int* output = new int[n];
    int count[10] = {0};
    for (int i = 0; i < n; i++) {
        int index = (arr[i] / exp) % 10;
        count[index]++;
    }
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        int index = (arr[i] / exp) % 10;
        output[count[index] - 1] = arr[i];
        count[index]--;
    }
    for (int i = 0; i < n; i++) {
        arr[i] = output[i];
    }
}
void radixSort(int arr[], int n) {
    if (n <= 0) return;
    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) max_val = arr[i];
    }
    for (int exp = 1; max_val / exp > 0; exp *= 10) {
        countingSortForRadix(arr, n, exp);
    }
}
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    if (n <= 0) {
        cout << "Array size must be greater than 0." << endl;
        return 1;
    }
    int* arr = new int[n];
   cout << "Enter " << n << " non-negative integers separated by spaces:\n";
```

```
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "\nOriginal array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << "\n";
    radixSort(arr, n);
    cout << "Sorted array:   ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << "\n";
    return 0;
}
```

Output :



3. Buket Sort :

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int a[100];
    int bucket[10][100];
    int count[10] = {0};
```

```
int n;

cout << "Enter number of elements: ";
cin >> n;

cout << "Enter elements (0-99): ";
for(int i = 0; i < n; i++)
{
   cin >> a[i];
}

// Put elements into buckets
for(int i = 0; i < n; i++)
{
   int index = a[i] / 10;
   bucket[index][count[index]] = a[i];
   count[index]++;
}

// Sort each bucket
for(int i = 0; i < 10; i++)
{
   for(int j = 0; j < count[i] - 1; j++)
   {
      for(int k = 0; k < count[i] - j - 1; k++)
      {
         if(bucket[i][k] > bucket[i][k + 1])
         {
            int temp = bucket[i][k];
            bucket[i][k] = bucket[i][k + 1];
            bucket[i][k + 1] = temp;
         }
      }
   }
}

// Combine buckets
int index = 0;

for(int i = 0; i < 10; i++)
{
   for(int j = 0; j < count[i]; j++)
   {
```

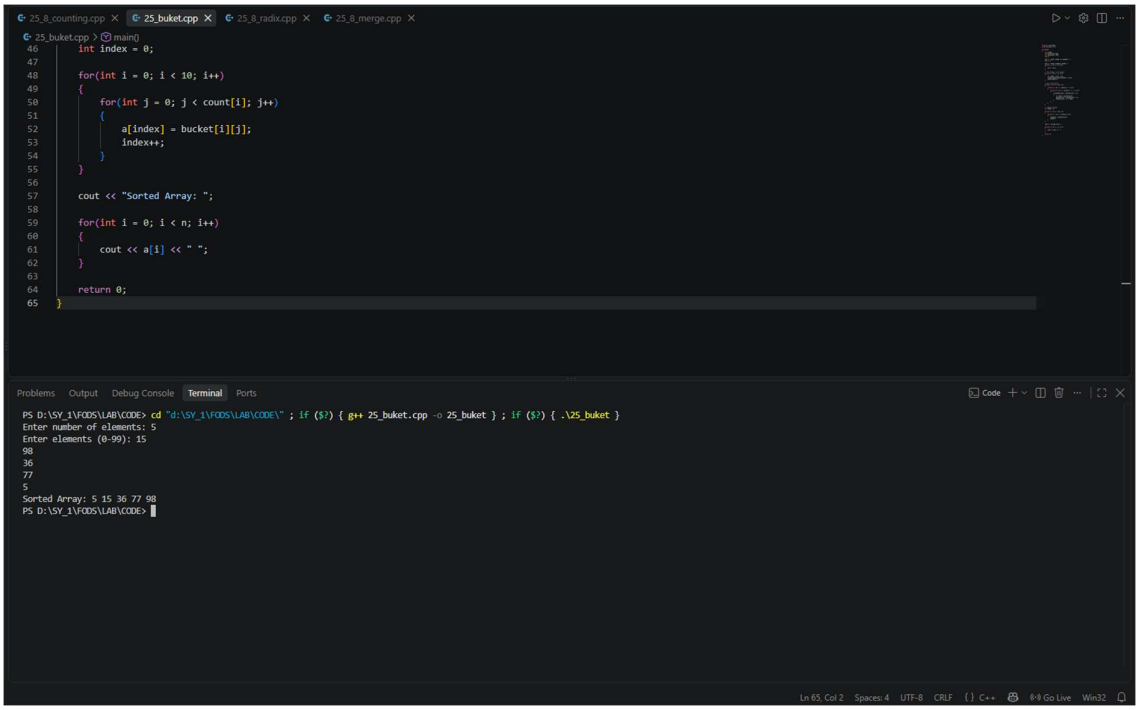
```
            a[index] = bucket[i][j];
            index++;
        }
    }

    cout << "Sorted Array: ";

    for(int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }

    return 0;
}
```

Output :



4. Merge Sort :

Code :

```
#include <iostream>
using namespace std;

void merge(int a[], int low, int mid, int high)
{
    int temp[100];

    int i = low;
    int j = mid + 1;
```

```
    int k = 0;

    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }
        k++;
    }

    while(i <= mid)
    {
        temp[k] = a[i];
        i++;
        k++;
    }

    while(j <= high)
    {
        temp[k] = a[j];
        j++;
        k++;
    }

    for(i = low, k = 0; i <= high; i++, k++)
    {
        a[i] = temp[k];
    }
}

void mergeSort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;
```

```
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int a[100];
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
    }

    mergeSort(a, 0, n - 1);

    cout << "Sorted Array: ";

    for(int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }

    return 0;
}
```

Output :



❖ Comparison With Comparison Based Sorting Algorithm:

Feature	Counting Sort	Radix Sort	Bucket Sort	Merge Sort	Quick Sort
Type	Non-Comparison	Non-Comparison	Non-Comparison	Comparison	Comparison
Best Case	O(n + k)	O(d(n + k))	O(n + k)	O(n log n)	O(n log n)
Average Case	O(n + k)	O(d(n + k))	O(n + k)	O(n log n)	O(n log n)
Worst Case	O(n + k)	O(d(n + k))	$O(n^2)$	O(n log n)	$O(n^2)$
Space Complexity	O(n + k)	O(n + k)	O(n + k)	O(n)	O(log n)
Stable	Yes	Yes	Yes*	Yes	No
Best Used For	Small range of integers	Large numbers	Uniformly distributed data	General-purpose sorting	Fast in-memory sorting